A Draft Recommendation Agent for HoK

Zitong Wang 22207273
zitong.wang@ucdconnect.ie

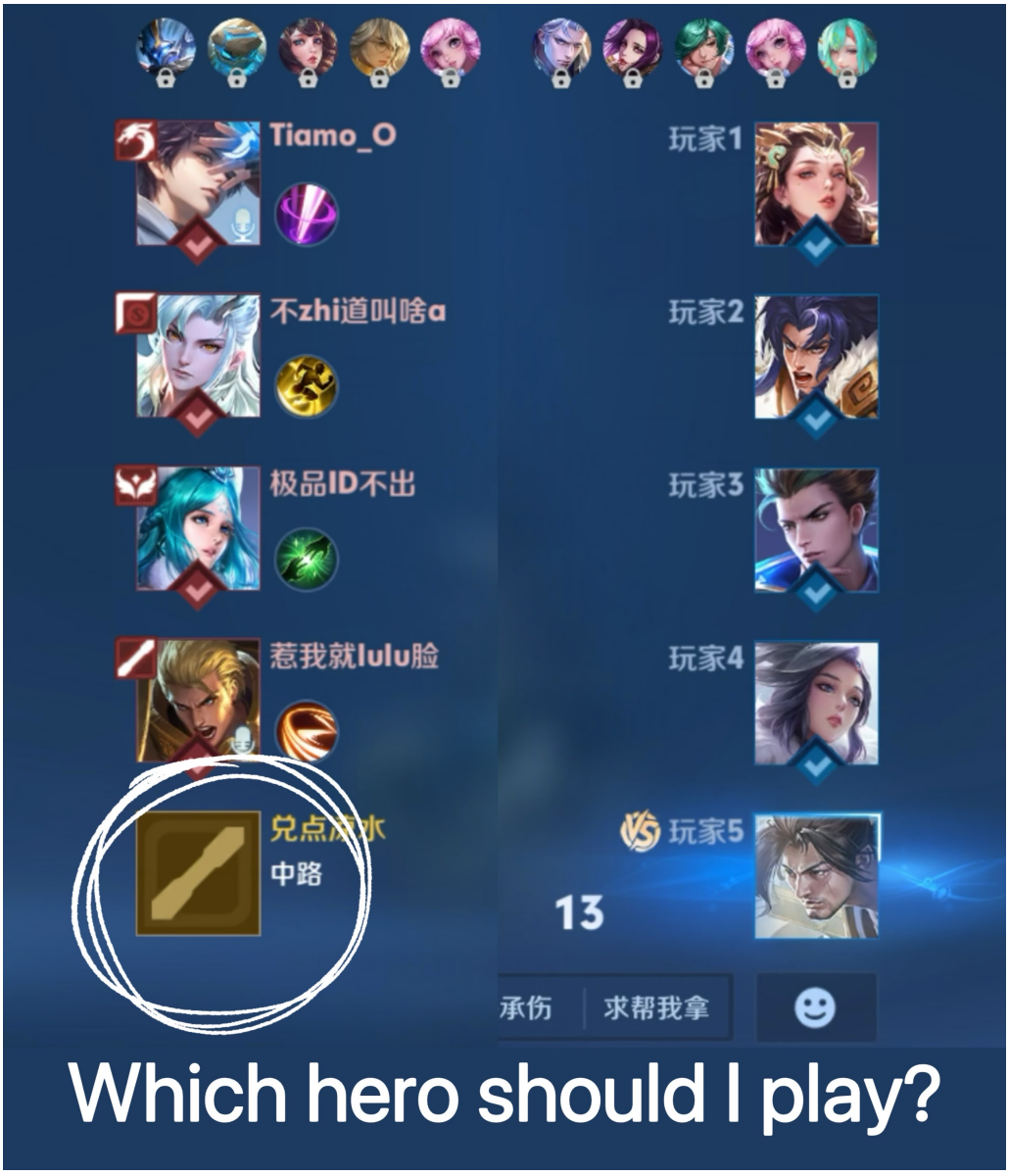


Figure 1: Project splash

Contents

1 Introduction **1**

2 Assistant Design: A Schematic View **1**

3 Added Value: More than Mere ChatGPT or basic LLM **2**

3.1 Tool Usage 2

3.1.1 Core Capabilities and API Integration 2

3.1.2 Design Constraints and System Rationale 3

4 Outside Knowledge: Curated Data Sources **3**

5 Worked Examples **4**

5.1 Example 1: Baseline output from ChatGPT 4

5.2 Example 2: Output from my agent on the same lineup 5

5.3 Example 3: My agent with Code Interpreter disabled 6

5.4 Example 4: Locked-hero mode (user already selected a hero) 7

5.5 Discussion 7

6 Summary and Conclusions **8**

7 Acknowledgements **8**

8 References **8**

List of Figures

1 Project splash 1

2 In-game draft phase in *HoK* 1

3 Overall architecture of the draft recommendation agent 2

4 Example of detecting official files and API calls using browser dev tool . . . 4

5 ChatGPT output for the baseline draft scenario 5

6 My agent output for the same draft scenario 6

7 Agent output with Code Interpreter disabled 7

8 Locked-hero mode: user hero vs lane matchup analysis 7

1 Introduction

Honor of Kings (*HoK*) is one of the most popular mobile MOBA games in China. It is a 5v5 strategy game focused on lane control, objectives, and teamfights.

A key decision happens before the match starts: the draft phase. During this stage, players choose heroes based on limited information, including teammates' picks, enemy picks, and their own assigned lane. A poor draft choice can create major problems for the whole match, such as weak frontline, lack of crowd control, or bad lane matchup.



Figure 2: In-game draft phase in *HoK*

The goal of this project is to build a technical draft coach agent that provides hero recommendations during the draft phase and concise macro-level guidance based on lane matchup, team composition, and current meta conditions. The agent is built using official and reliable game data sources to ensure grounded and consistent decision-making.

2 Assistant Design: A Schematic View

The agent goal is to produce lane-aware, evidence-grounded recommendations with stable runtime behavior in a notebook setting. To achieve this, the system leverages Responses API multi-turn tool orchestration (function calls and function-call outputs), optional file-search retrieval, and controlled retry/backoff for robustness. At the same time, some features are intentionally constrained: Code Interpreter is not part of the default main loop, and heavy parallel tool execution is avoided, because this project prioritizes lower token pressure, easier debugging, and reproducible outputs over maximum tool-call throughput.

Figure 3 summarizes this design choice: a lightweight, tool-grounded pipeline that converts structured match inputs into auditable recommendation outputs.

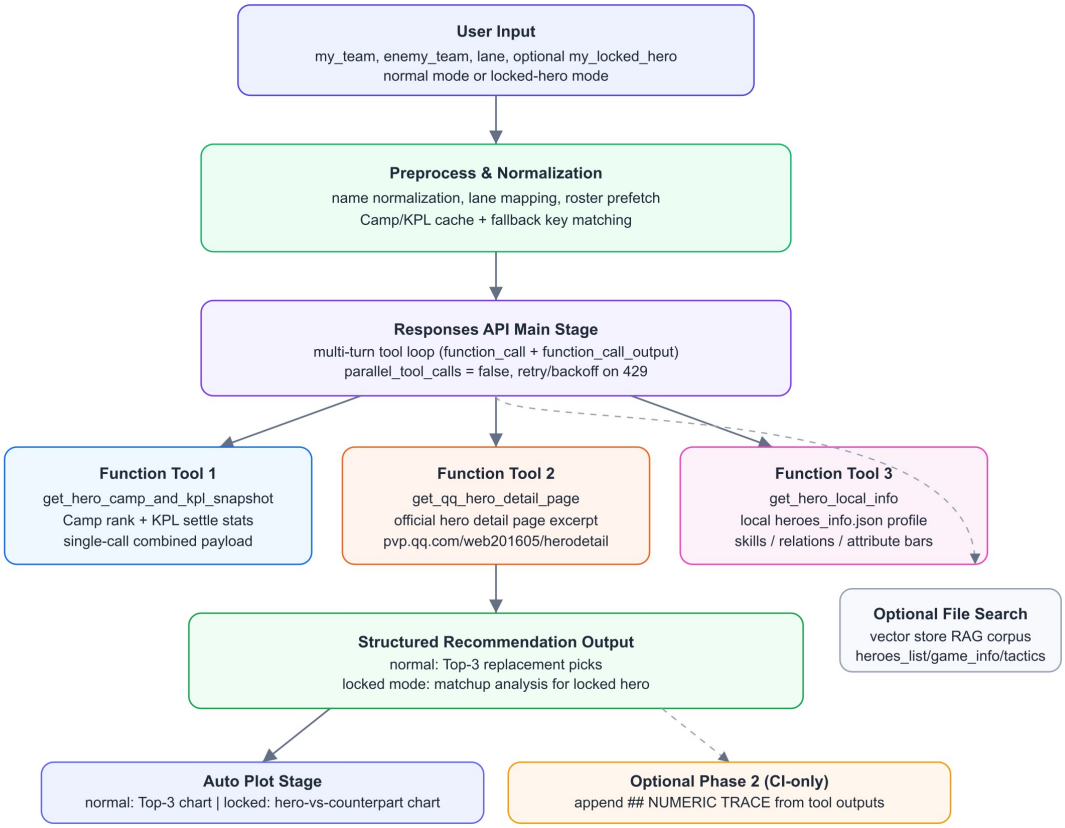


Figure 3: Overall architecture of the draft recommendation agent

3 Added Value: More than Mere ChatGPT or basic LLM

Compared with vanilla ChatGPT, this assistant is designed for a narrower but more practical draft-coaching task: it combines real-time external statistics (Camp/KPL), official hero-detail extraction, curated local heroes info, and optional file-search retrieval inside a structured Responses API tool loop, so recommendations are grounded in explicit data rather than model memory alone; it also enforces output structure (decision block), supports mode switching (Top-3 replacement mode vs locked-hero matchup mode), and can produce optional numeric traces, which together make the system more reliable, auditable, and decision-oriented than a general chat response for the same lineup input.

3.1 Tool Usage

3.1.1 Core Capabilities and API Integration

The implementation is powered by the OpenAI Responses API and integrates multiple tool-based components:

- **Iterative tool execution loop:** supports function calling cycles through `function_call` and `function_call_output`, enabling real-time data retrieval and reasoning.

- **Game data tools:** include Camp + KPL statistical snapshots, official hero detail extraction, and local heroes info retrieval.
- **Optional retrieval layer:** a vector-store-based file search module provides additional contextual drafting knowledge.
- **Robust execution control:** retry and backoff mechanisms are used to handle API rate limits and ensure stable notebook execution.
- **Two-stage reasoning support:** optional generation of a structured ## NUMERIC TRACE for deeper analysis without affecting the main recommendation output.

3.1.2 Design Constraints and System Rationale

Several design choices were made to ensure stability, interpretability, and efficiency:

- **No parallel tool execution:** parallel calls are disabled to avoid token spikes and improve debugging traceability.
- **Separation of reasoning stages:** numerical trace generation is decoupled from the main recommendation output to keep responses lightweight.
- **Data-driven decision making:** recommendations are grounded in live statistics and structured tool outputs rather than implicit model memory.
- **Robustness mechanisms:** normalization, caching, prefetching, and fallback parsing ensure stable behavior across different input conditions.

4 Outside Knowledge: Curated Data Sources

To provide reliable data support, the agent uses external game data collected from official sources through browser developer tools and structured file extraction. A lightweight pipeline was then built to clean, normalize, and connect these sources for runtime use.

The system mainly relies on three data sources:

- **Official Hero Detail Page Endpoint** (*accessed through browser developer tools and network inspection; used for hero text extraction*)
  `https://pvp.qq.com/web201605/herodetail/`
- **Camp Hero Ranking API** (*internal endpoint used for ranked hero statistics*)
  `https://api-camp.honorofkings.com/game/hero/getranklist`

- **KPL Hero List API** (*professional league statistics endpoint*)
  `https://prod.comp.smoba.qq.com/leaguesite/league/hero/settle_list/open`

Data scale in this project: The system operates on approximately 130 hero entries from the full roster dataset, combined with a smaller curated subset of detailed hero profiles (used for RAG-style retrieval), together with 3 external live data endpoints covering ladder statistics, professional league performance, and official hero descriptions.

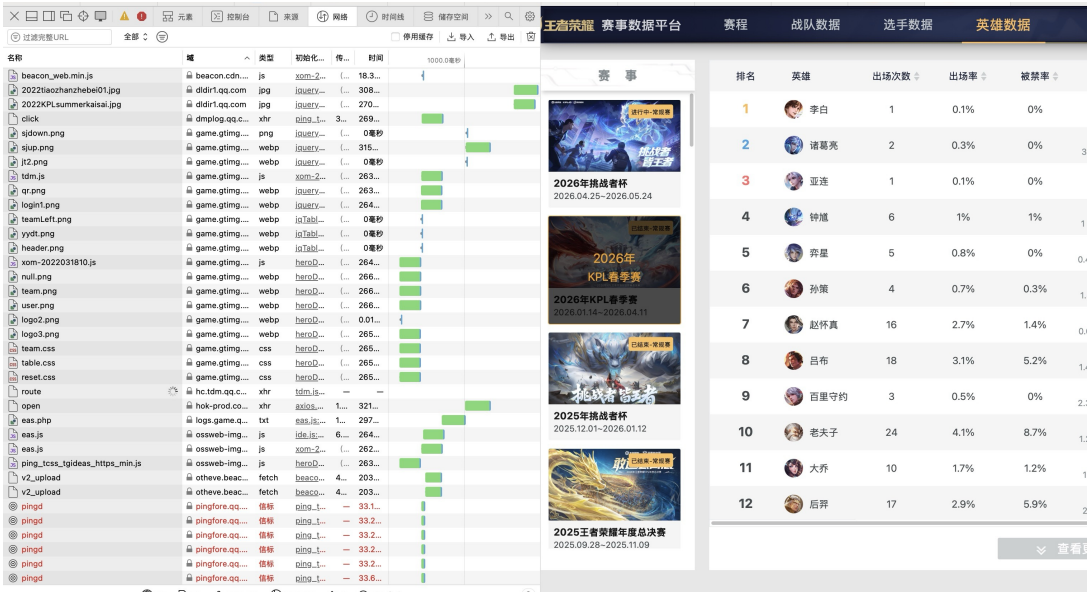


Figure 4: Example of detecting official files and API calls using browser dev tool

In addition, static hero metadata and tactical text materials were extracted and stored locally. These support retrieval and reduce repeated external requests.

To make multi-source data usable inside one agent loop, I implemented canonical name normalization based on cname, cache maps for Camp/KPL lookups, roster prefetch for efficiency, and robust output parsing for different model response formats. Prompt consistency rules were also added to prevent contradictions such as reporting "no data" when valid numerical fields already exist.

These sources are exposed in the notebook through three function tools: `get_hero_camp_and_kpl_snapshot`, `get_qq_hero_detail_page`, and `get_hero_local_info`.

5 Worked Examples

5.1 Example 1: Baseline output from ChatGPT

Below is the output generated by ChatGPT for one draft scenario. It can be seen that although I asked the model to "answer with related data support" and it did provide a

detailed analysis, it did not actually include any official data to support its claims.

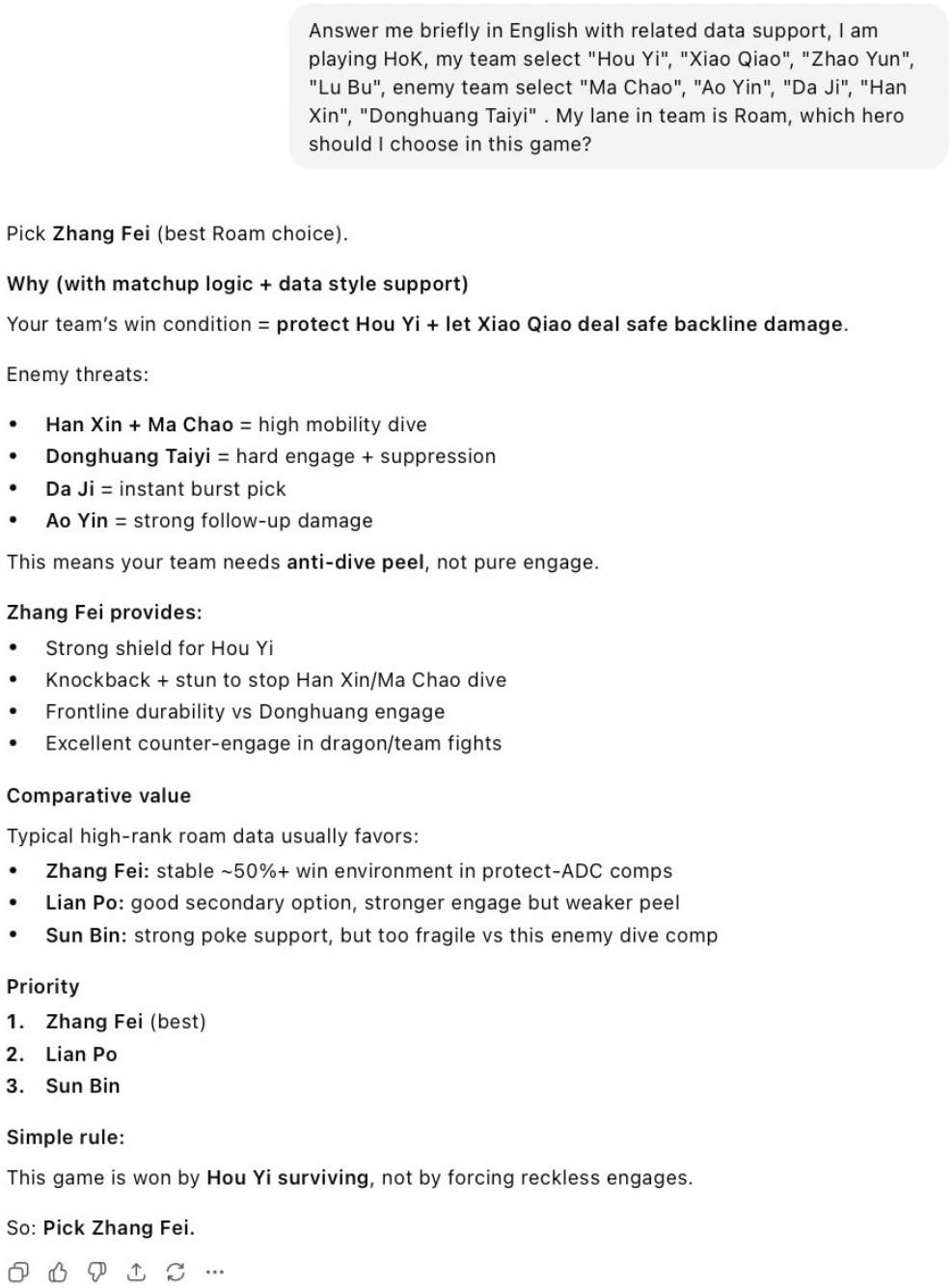


Figure 5: ChatGPT output for the baseline draft scenario

5.2 Example 2: Output from my agent on the same lineup

Using the same lineup input, my agent produces a more structured and tool-based answer.

In the following output, I enabled the detailed-numeric-explanation mode and used the

Code Interpreter to compute the data. It can be seen that my agent reaches its conclusion by jointly processing information from Camp data, KPL data, and heroes information. The final result is the same as ChatGPT's optimal choice. However, the second and third ranked alternatives differ between the two outputs. With additional data support, my agent's response is more convincing than ChatGPT's.

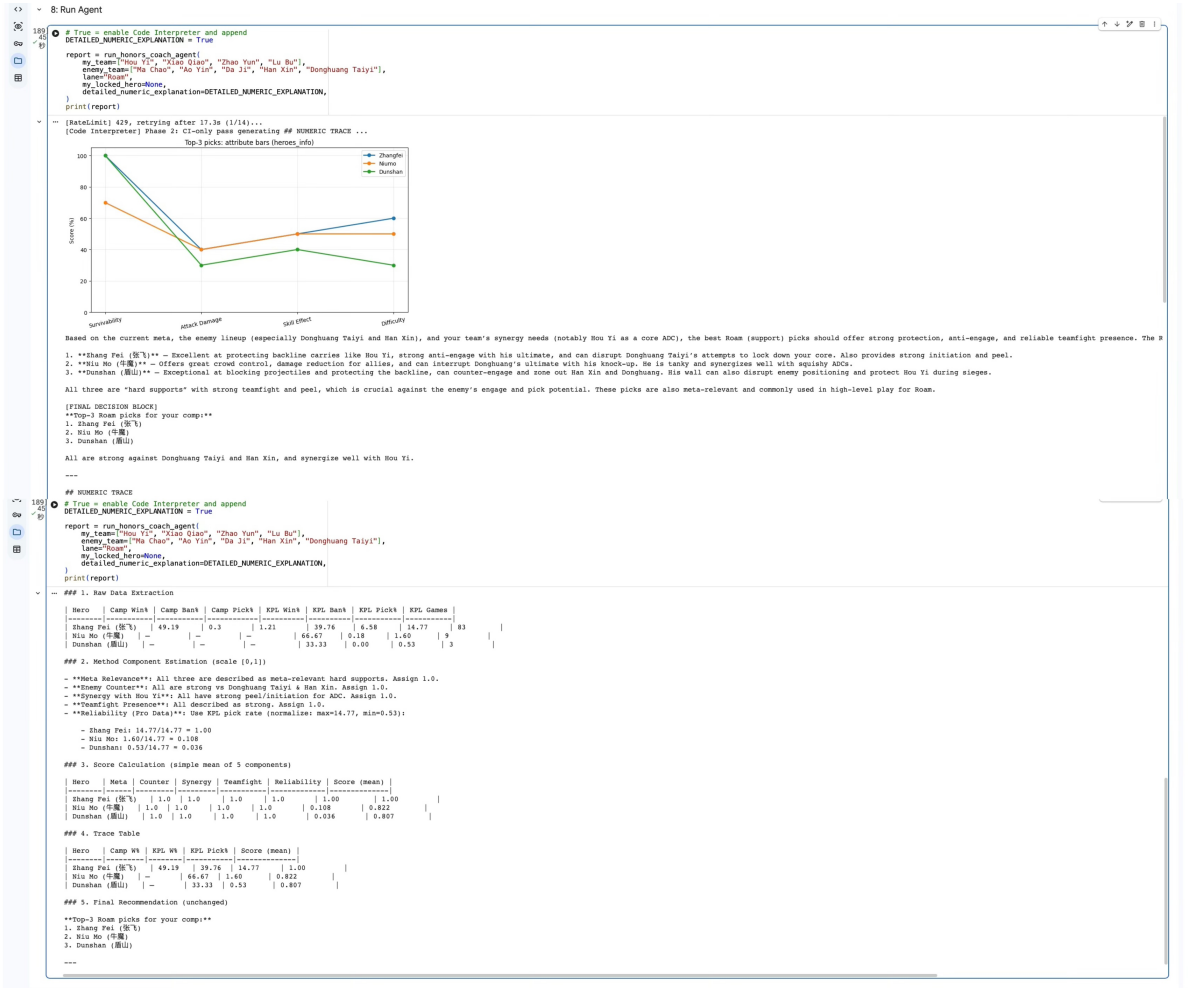


Figure 6: My agent output for the same draft scenario

5.3 Example 3: My agent with Code Interpreter disabled

My agent also supports turning off Code Interpreter output. In this mode, it still returns structured recommendations without the extra numeric trace block.

Even with Code Interpreter output disabled, the assistant still grounds its main recommendation on the same retrieved and tool-fetched evidence (e.g., Camp/KPL snapshots, official hero details, and local heroes info). The difference is that it skips the extra post-hoc numeric-trace computation stage, so the response remains data-informed while avoiding additional trace-generation overhead.

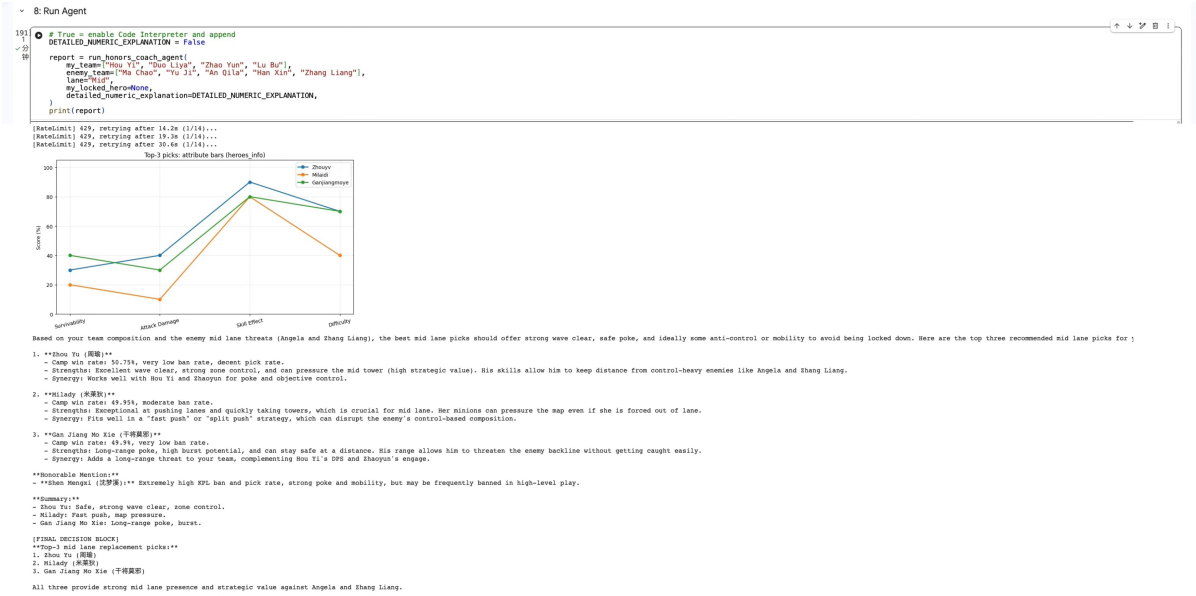


Figure 7: Agent output with Code Interpreter disabled

5.4 Example 4: Locked-hero mode (user already selected a hero)

My agent additionally supports a locked-hero mode. If the user has already selected a hero, the agent analyzes that hero against the same-lane opponent instead of generating a generic Top-3 replacement list.

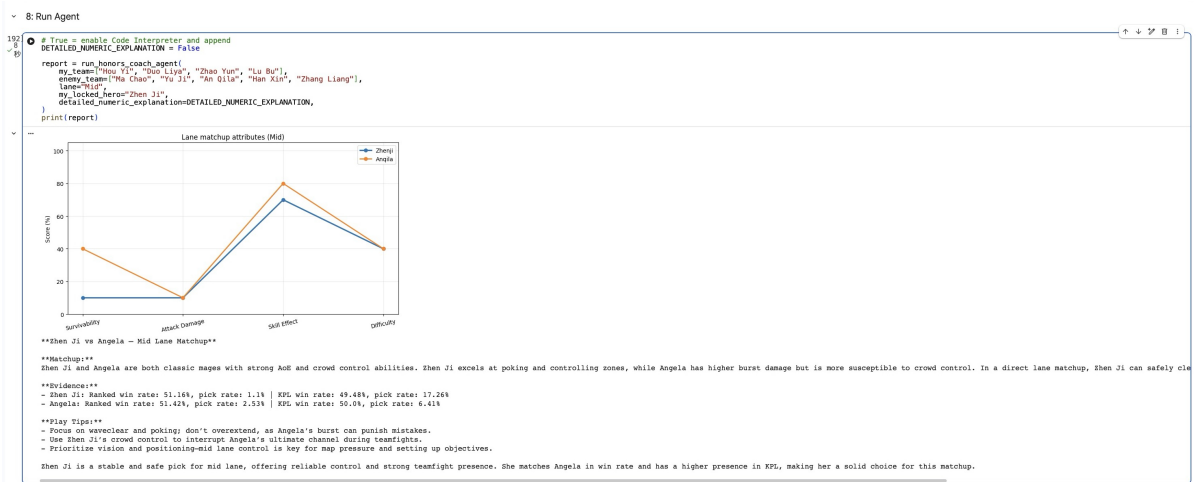


Figure 8: Locked-hero mode: user hero vs lane matchup analysis

5.5 Discussion

Across these examples, the main added value of my agent is:

- more structured output for decision-making in draft phase;
- direct integration of external statistics and curated local data;
- flexible runtime modes (with/without CI, normal mode, locked-hero mode).

6 Summary and Conclusions

This project implements a practical draft recommendation agent for *Honor of Kings* using the Responses API, tool calling, optional file-search retrieval, and optional CI-based numeric tracing.

Compared with a vanilla chat baseline, the assistant provides stronger structure, clearer evidence paths, and mode-aware behavior (normal replacement mode and locked-hero matchup mode), making it more suitable for real pre-game pick decisions.

Future improvements include: stronger alias resolution across all endpoints, more automated evaluation logs, and a more deterministic scoring layer for long-run consistency.

7 Acknowledgements

I acknowledge the official OpenAI and Tencent developer documentation cited in the References section, which were used as technical resources for implementation and verification. I confirm that this report represents my own independent work. All external sources, including APIs, webpages, and technical materials, are properly cited in-text and in the References section. Any directly quoted content is clearly attributed.

8 References

[1] OpenAI, *Responses API Documentation*, 2026.
`https://platform.openai.com/docs/guides/migrate-to-responses`

[2] OpenAI, *Tools, Function Calling, and File Search Documentation*, 2026.
`https://platform.openai.com/docs/guides/tools`

[3] Tencent Games, *Honor of Kings KPL Data Page*.
`https://pvp.qq.com/matchdata/heroData.html?league_id=20260001`

[4] Tencent Games, *Honor of Kings Hero List Page*.
`https://pvp.qq.com/web201605/herolist.shtml`

[5] HOK Camp, *Hero Hot List*.
`https://camp.honorofkings.com/h5/app/index.html#/hero-hot-list`